



Programming Fundamentals

Introduction to Iteration

Programming Fundamentals Team

SETU

2026-09-28



Three Fundamental Ideas Three Fundamental Ideas Three Fundamental Ideas Three Fundamental Ideas Think About Everyday Life Think About Everyday Life

Think About Everyday Life Think About Everyday Life Think About Everyday Life Two Types of Loop Two Types of Loop Which Loop? Which Loop? Which Loop?

Trace It Trace It What Could Possibly Go Wrong? What Could Possibly Go Wrong? What Could Possibly Go Wrong? What Could Possibly Go Wrong? Infinite Loops

Infinite Loops Infinite Loops Bringing Our Skills Together Bringing Our Skills Together Bringing Our Skills Together Our Programming Toolkit Our Programming Toolkit

Our Programming Toolkit Today's Focus Today's Focus





1. Where Are We Now?

1. Where Are We Now?

Over the last few weeks, we have been building up our Python toolkit.



1. Where Are We Now?

1. Where Are We Now?

Over the last few weeks, we have been building up our Python toolkit.

We can now:

- store information using **variables**
- work with different **data types**
- make decisions using **selection**
- organise our code using **functions**



1. Where Are We Now?

1. Where Are We Now?

Over the last few weeks, we have been building up our Python toolkit.

We can now:

- store information using **variables**
- work with different **data types**
- make decisions using **selection**
- organise our code using **functions**

We can already write programs that take input, make decisions and produce useful results.

Variables allow us to **store information** while our program is running.

```
name = "Charlie"  
age = 4  
weight = 28.5  
vaccinated = True
```



2. Variables – Remember These?

Our Programming Toolkit Today's Focus Today's Focus

```
name = "Charlie"  
age = 4  
weight = 28.5  
vaccinated = True
```

Different values can have different **data types**.

- str
- int
- float
- bool



2. Variables – Remember These?

Our Programming Toolkit

```
name = "Charlie"  
age = 4  
weight = 28.5  
vaccinated = True
```

Different values can have different **data types**.

- str
- int
- float
- bool

And variables can change:



2. Variables – Remember These?

```
age = age + 1
```



3. Selection – Making Decisions

3. Selection – Making Decisions

We then learned how our programs can make decisions.

```
if age < 1:
    print("Puppy")
elif age < 8:
    print("Adult dog")
else:
    print("Senior dog")
```



3. Selection – Making Decisions

3. Selection – Making Decisions

We then learned how our programs can make decisions.

```
if age < 1:
    print("Puppy")
elif age < 8:
    print("Adult dog")
else:
    print("Senior dog")
```

Selection asks a question:

Is this condition True or False?



3. Selection – Making Decisions

3. Selection – Making Decisions

We then learned how our programs can make decisions.

```
if age < 1:
    print("Puppy")
elif age < 8:
    print("Adult dog")
else:
    print("Senior dog")
```

Selection asks a question:

Is this condition True or False?

The answer determines which code runs.



4. Functions – Organising Our Code

4. Functions – Organising Our Code

Functions allow us to give a piece of code a job.

```
def calculate_cost(days, daily_price):  
    cost = days*daily_price  
    return cost
```



6. Functions – Organising Our Code

6. Functions – Organising Our Code

Functions allow us to give a piece of code a job.

```
def calculate_cost(days, daily_price):  
    cost = days*daily_price  
    return cost  
  
total = calculate_cost(3, 25.0)
```



6. Functions – Organising Our Code

6. Functions – Organising Our Code

Functions allow us to give a piece of code a job.

```
def calculate_cost(days, daily_price):  
    cost = days*daily_price  
    return cost
```

```
total = calculate_cost(3, 25.0)
```

Functions help us:

- organise our program
- avoid unnecessary repetition
- reuse code
- break a larger problem into smaller problems



7. We Can Already Do Quite A Lot

Imagine a simple Dog Day Care program.

```
name = input("Dog's name: ")
age = int(input("Dog's age: "))

if age < 1:
    print(name, "is a puppy")
else:
    print(name, "is not a puppy")
```



7. We Can Already Do Quite A Lot



9. We Can Already Do Quite A Lot

Imagine a simple Dog Day Care program.

```
name = input("Dog's name: ")
age = int(input("Dog's age: "))

if age < 1:
    print(name, "is a puppy")
else:
    print(name, "is not a puppy")
```

This program:

- gets input
- stores values
- makes a decision



9. We Can Already Do Quite A Lot

- displays a result



9. We Can Already Do Quite A Lot

Imagine a simple Dog Day Care program.

```
name = input("Dog's name: ")
age = int(input("Dog's age: "))

if age < 1:
    print(name, "is a puppy")
else:
    print(name, "is not a puppy")
```

This program:

- gets input
- stores values
- makes a decision



9. We Can Already Do Quite A Lot

- displays a result

But what happens if we want to do it again?



10. A New Problem...

Suppose we want our program to print:

Welcome!

Welcome!

Welcome!

Welcome!

Welcome!



10. A New Problem...



12. A New Problem...

Suppose we want our program to print:

Welcome!

Welcome!

Welcome!

Welcome!

Welcome!

We *could* write:

```
print("Welcome!")
```

```
print("Welcome!")
```

```
print("Welcome!")
```

```
print("Welcome!")
```

```
print("Welcome!")
```



12. A New Problem...



12. A New Problem...

Suppose we want our program to print:

Welcome!
Welcome!
Welcome!
Welcome!
Welcome!

We *could* write:

```
print("Welcome!")  
print("Welcome!")  
print("Welcome!")  
print("Welcome!")  
print("Welcome!")
```



12. A New Problem...

But what if we wanted it **100 times?**



13. Or Imagine This...

```
password = input("Enter password: ")
```

```
if password == "python":  
    print("Access granted")  
else:  
    print("Incorrect password")
```



13. Or Imagine This...



15. Or Imagine This...

```
password = input("Enter password: ")
```

```
if password == "python":  
    print("Access granted")  
else:  
    print("Incorrect password")
```

What happens if the password is wrong?



15. Or Imagine This...



15. Or Imagine This...

```
password = input("Enter password: ")
```

```
if password == "python":  
    print("Access granted")  
else:  
    print("Incorrect password")
```

What happens if the password is wrong?

The program ends.



15. Or Imagine This...



15. Or Imagine This...

```
password = input("Enter password: ")
```

```
if password == "python":  
    print("Access granted")  
else:  
    print("Incorrect password")
```

What happens if the password is wrong?

The program ends.

But what if we want to say:

Keep asking until the user enters the correct password.



15. Or Imagine This...

We need a way of **repeating code**.



16. Iteration

16. Iteration

Iteration means repeating a set of instructions.



18. Iteration

18. Iteration

Iteration means repeating a set of instructions.

We often call this a **loop**.



18. Iteration

18. Iteration

Iteration means repeating a set of instructions.

We often call this a **loop**.

Instead of writing the same code again and again...

```
print("Hello")
print("Hello")
print("Hello")
```

...we can tell Python to repeat it for us.



18. Iteration

18. Iteration

Iteration means repeating a set of instructions.

We often call this a **loop**.

Instead of writing the same code again and again...

```
print("Hello")  
print("Hello")  
print("Hello")
```

...we can tell Python to repeat it for us.

Iteration is the third major building block of programming.



19. Three Fundamental Ideas

19. Three Fundamental Ideas

Sequence → Selection → Iteration



21. Three Fundamental Ideas

21. Three Fundamental Ideas

Sequence → Selection → Iteration

Sequence

Do instructions in order.



21. Three Fundamental Ideas

21. Three Fundamental Ideas

Sequence → Selection → Iteration

Sequence

Do instructions in order.

Selection

Decide which instructions to perform.



21. Three Fundamental Ideas

21. Three Fundamental Ideas

Sequence → Selection → Iteration

Sequence

Do instructions in order.

Selection

Decide which instructions to perform.

Iteration

Repeat instructions.



21. Three Fundamental Ideas

21. Three Fundamental Ideas

Sequence → Selection → Iteration

Sequence

Do instructions in order.

Selection

Decide which instructions to perform.

Iteration

Repeat instructions.

These three ideas appear in almost every program we write.



22. Think About Everyday Life

22. Think About Everyday Life

We use repetition all the time.



25. Think About Everyday Life

25. Think About Everyday Life

We use repetition all the time.

Brushing your teeth

Repeat brushing movements until enough time has passed.



25. Think About Everyday Life

25. Think About Everyday Life

We use repetition all the time.

Brushing your teeth

Repeat brushing movements until enough time has passed.

Washing dishes

While there are dirty dishes, wash another dish.



25. Think About Everyday Life

25. Think About Everyday Life

We use repetition all the time.

Brushing your teeth

Repeat brushing movements until enough time has passed.

Washing dishes

While there are dirty dishes, wash another dish.

Taking attendance

For each student, record whether they are present.



25. Think About Everyday Life

25. Think About Everyday Life

We use repetition all the time.

Brushing your teeth

Repeat brushing movements until enough time has passed.

Washing dishes

While there are dirty dishes, wash another dish.

Taking attendance

For each student, record whether they are present.

Programming works in much the same way.



26. Two Types of Loop

26. Two Types of Loop

In Python, we will look at two main ways of repeating code.



29. Two Types of Loop

29. Two Types of Loop

In Python, we will look at two main ways of repeating code.

while loop

Repeat while a condition is True.

```
while condition:  
    repeat this
```

for loop

Repeat for a collection or sequence of values.

```
for value in sequence:  
    repeat this
```



30. Which Loop?

30. Which Loop?

Don't worry about choosing between them yet.



34. Which Loop?

34. Which Loop?

Don't worry about choosing between them yet.

A useful starting point is:

while → repeat while something is true

for → repeat for a set of values



34. Which Loop?

34. Which Loop? [Our Programming Toolkit](#) [Today's Focus](#) [Today's Focus](#)

Don't worry about choosing between them yet.

A useful starting point is:

while → repeat while something is true

for → repeat for a set of values

This week:

- **Day 1:** while loops
- **Day 2:** for loops



35. Our First while Loop

35. Our First while Loop

```
count = 1
```

```
while count <= 5:
    print(count)
    count = count + 1
```



39. Our First while Loop

39. Our First while Loop

```
count = 1
```

```
while count <= 5:
    print(count)
    count = count + 1
```

What do you think this program will display?



39. Our First while Loop

39. Our First while Loop

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count = count + 1
```

What do you think this program will display?

1
2
3
4
5



40. Look Closely...

40. Look Closely...

```
count = 1
```

```
while count <= 5:
    print(count)
    count = count + 1
```

We already understand nearly everything here!



40. Look Closely...



41. Look Closely...

41. Look Closely...

```
count = 1
```

```
while count <= 5:
    print(count)
    count = count + 1
```

We already understand nearly everything here!

```
count = 1
```

Variable



41. Look Closely...



41. Look Closely...

41. Look Closely...

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count = count + 1
```

We already understand nearly everything here!

```
count = 1
```

Variable

```
count <= 5
```

Boolean condition



41. Look Closely...



41. Look Closely...

41. Look Closely...

```
count = 1
```

```
while count <= 5:
    print(count)
    count = count + 1
```

We already understand nearly everything here!

```
count = 1
```

Variable

```
count <= 5
```

Boolean condition



41. Look Closely...

```
count = count + 1
```

Updating a variable



42. The New Bit

The only really new idea is:

```
while count <= 5:
```



44. The New Bit

The only really new idea is: Our Programming Toolkit Today's Focus Today's Focus

```
while count <= 5:
```

Python checks the condition.

If it is True:

- run the indented code
- go back
- check the condition again



44. The New Bit

The only really new idea is:

```
while count <= 5:
```

Python checks the condition.

If it is True:

- run the indented code
- go back
- check the condition again

If it is False:

- leave the loop
- continue with the rest of the program



45. Trace It

45. Trace It

```
count = 1
```

```
while count <= 5:
    print(count)
    count = count + 1
```



47. Trace It

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

count	count <= 5	output
1	True	1
2	True	2
3	True	3
4	True	4
5	True	5
6	False	Stop



48. What Could Possibly Go Wrong?

48. What Could Possibly Go Wrong?

Look carefully at this:

```
count = 1

while count <= 5:
    print(count)
```



51. What Could Possibly Go Wrong?

Look carefully at this:

```
count = 1
```

```
while count <= 5:  
    print(count)
```

What will happen?



51. What Could Possibly Go Wrong?

Look carefully at this:

```
count = 1

while count <= 5:
    print(count)
```

What will happen?

count starts at 1.

count <= 5 is True.

But...



51. What Could Possibly Go Wrong?

Look carefully at this:

```
count = 1
```

```
while count <= 5:  
    print(count)
```

What will happen?

count starts at 1.

count <= 5 is True.

But...

count never changes!



52. Infinite Loops

The previous program will keep printing:

```
1
1
1
1
1
...
```



54. Infinite Loops

54. Infinite Loops

The previous program will keep printing: Today's Focus Today's Focus

```
1
1
1
1
1
...
```

This is called an **infinite loop**.



54. Infinite Loops

The previous program will keep printing:

```
1
1
1
1
1
...
```

This is called an **infinite loop**.

When using a `while` loop, always ask:

What will eventually make my condition False?



55. Bringing Our Skills Together

Iteration doesn't replace anything we have learned.

It works **with** it.



55. Bringing Our Skills Together



56. Bringing Our Skills Together

Iteration doesn't replace anything we have learned.

It works **with** it.

```
def check_age(age):  
    if age >= 18:  
        return "Adult"  
    else:  
        return "Under 18"  
  
again = "yes"  
  
while again == "yes":  
    age = int(input("Enter age: "))  
    print(check_age(age))
```



56. Bringing Our Skills Together

```
again = input("Go again? yes/no: ")
```



56. Bringing Our Skills Together

56. Bringing Our Skills Together

Iteration doesn't replace anything we have learned.

It works **with** it.

```
def check_age(age):  
    if age >= 18:  
        return "Adult"  
    else:  
        return "Under 18"  
  
again = "yes"  
  
while again == "yes":  
    age = int(input("Enter age: "))  
    print(check_age(age))
```



56. Bringing Our Skills Together

```
again = input("Go again? yes/no: ")
```

Can you spot:

- a variable?
- selection?
- a function?
- a parameter?
- a return value?
- iteration?



57. Our Programming Toolkit

57. Our Programming Toolkit

We now have four very powerful tools.



60. Our Programming Toolkit

60. Our Programming Toolkit

We now have four very powerful tools.

Concept	What does it let us do?
Variables	Remember information
Selection	Make decisions
Functions	Organise and reuse code
Iteration	Repeat code



60. Our Programming Toolkit

60. Our Programming Toolkit

We now have four very powerful tools.

Concept	What does it let us do?
Variables	Remember information
Selection	Make decisions
Functions	Organise and reuse code
Iteration	Repeat code

The real power comes from combining them.



61. Today's Focus

61. Today's Focus

Our Programming Toolkit

Today's Focus

Today we are going to learn how to use `while` loops to:

- repeat code while a condition is true
- use a counter
- update variables inside a loop
- repeat user input
- use selection inside a loop
- avoid infinite loops
- trace a loop by hand



63. Today's Focus

63. Today's Focus

Our Programming Toolkit Today's Focus

Today we are going to learn how to use `while` loops to:

- repeat code while a condition is true
- use a counter
- update variables inside a loop
- repeat user input
- use selection inside a loop
- avoid infinite loops
- trace a loop by hand

By the end, you should be able to look at a `while` loop and explain **exactly why it stops**.